

DAA - Assignment 3

Name: J NUSKA NATH.

Roll: 002311001003

Dept: IT

Year: UG3.

Q1) Write an algorithm to check that if a given list of numbers is sorted.

Ans) Let the list be L with n numbers.

Algorithm (to check for non-decreasing sort)

Input: List L of n numbers.

Output: True if sorted, False otherwise.

1. for $i \leftarrow 1$ to $n-1$ do
2. if $L[i] > L[i+1]$ then
3. return False
4. end if
5. end for
6. return True

Time Complexity

Every element is compared to the next once $\rightarrow O(n)$

Correctness proof (Invariant)

i) Invariant: At the start of each loop iteration, all elements up to $L[i]$ are in non decreasing order.

ii) Initialization: Before the first iteration ($i=1$), no comparisons have been made and the invariant is vacuously true.

iii) Maintenance: If $L[i] \leq L[i+1]$, the invariant remains true for the next iteration.

iv) Termination: If any out-of-order pair is found, the algorithm stops and correctly returns False. If none are found, all pairs are in order, so the list is sorted.

Q2) Consider the following list of numbers:

4, 2, 61, 45, 1, 7, 9, 19, 56, 21, 24

Write an algorithm to place 9 in its correct location
sorted position.

Ans) Strategy: Take the element 9, and insert it in a
way similar to Insertion sort.

Algorithm

Algorithm Place Nine (L, n)

Input: List L , Length n

Output: List with 9 moved to its correct sorted
position.

1. Find index k where $L[k] = 9$
2. $value \leftarrow 9$
3. $j \leftarrow k - 1$
4. while ($j \geq 1$ and $L[j] > value$) do
5. $L[j+1] \leftarrow L[j]$
6. $j \leftarrow j - 1$
7. end while
8. $L[j+1] \leftarrow value$

After execution the list becomes:

1, 2, 4, 7, 9, 61, 45, 19, 56, 21, 24.

Time Complexity:

At worst, shifting might ^{compare} ~~compare~~ with all previous
elements $\rightarrow O(n)$

Correctness

By moving 9 left until all larger ~~elements~~ are
shifted right, the element is positioned so that
~~everything~~ everything to its left is lesser than 9
and to its right is greater than 9.

83) Write an algorithm to 9 in first location, followed by all numbers less than 9, followed by all numbers greater than 9.

Ans) Algorithm

Partition On Nine (L, n)

Input: List L of n numbers

Output: Reordered list.

1. Create three empty lists: less, greater, Result
2. for $i \leftarrow 1$ to n do
3. if $L[i] < 9$ then
4. append $L[i]$ to less
5. else if $L[i] > 9$ then
6. append $L[i]$ to greater
7. end if
8. end for
9. $\text{Result} \leftarrow [9] + \text{less} + \text{greater}$
10. return Result.

Example:

Input: 4, 2, 61, 45, 1, 7, 9, 19, 56, 21, 24

Output: 9, 4, 2, 1, 7, 61, 45, ¹⁹56, 21, 24.

Analysis:

- Each input is examined once $\rightarrow$ time $O(n)$
- No sorting within subgroups — only grouping.

Correctness:

Every element less than 9 ends up immediately after the pivot.

All greater go after those.

The order within each subgroup stays the same (stable partition).

84) Let A and B are 32 digit numbers

a) Write an algorithm to add A and B

Ans) Algorithm:

Big Add (A, B)

Input: A, B with 32 digits each

output: C with up to 33 digits.

1. $Carry \leftarrow 0$
2. for $i \leftarrow 32$ down to 1 do
3. $sum \leftarrow A[i] + B[i] + carry$
4. $C[i+1] \leftarrow sum \bmod 10$
5. $carry \leftarrow sum \div 10$
6. end for
7. $C[1] \leftarrow carry$
8. return C.

Example:

if $A = 123\dots$, $B = 789\dots$, algorithm handles digit by digit addition with carry.

Complexity:

32 ~~steps~~ ~~each~~ digit steps $\rightarrow O(1)$ (constant bounded)

Correctness:

Each digit is processed with correct base-10 addition and carry propagation.

The final carry goes into the most significant digit.

5) Write an algorithm to multiply A and B.

Ans) This is multiplication via digit grid method.

Big Multiply (A, B)

Input: A[32] B[32]

Output: P[64] (product)

1. Initialize $P[1..64] \leftarrow 0$
2. for $i \leftarrow 32$ down to 1 do
3. ~~carry~~ $\leftarrow 0$
4. for ~~i~~ $j \leftarrow 32$ down to 1 do
5. $\text{temp} \leftarrow P[i+j] + A[i] * B[j] + \text{carry}$
6. $P[i+j] \leftarrow \text{temp} \bmod 10$
7. $\text{carry} \leftarrow \text{temp} \div 10$
8. end for.
- ~~9. for~~
9. ~~$P[i]$~~ $P[i] \leftarrow P[i] + \text{carry}$
10. end for
11. ~~return~~ return P.

Example:

if $A = 12$, $B = 34$, this produces 408.

Time Complexity

Nested loops of length 32 $\rightarrow O(32 \times 32) = O(1)$
(bounded)

Correctness

At each step we multiply digits, add into their appropriate place value, manage carry, and accumulate partial sums just like manual multiplication.